Paso 3 · Consultas del leaderboard

CREATE TABLE ventas (venta_id INT AUTO_INCREMENT PRIMARY KEY, cliente_id INT NOT NULL, ropa_id INT NOT NULL, -- Conecta con la tabla ropa fecha_venta DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP, FOREIGN KEY (cliente_id) REFERENCES clientes(cliente_id), FOREIGN KEY (ropa_id) REFERENCES ropa(ropa_id)) ENGINE=InnoDB;

SELECT

c.nombre_usuario,

c.email,

MAX(r.precio) AS compra_mas_cara

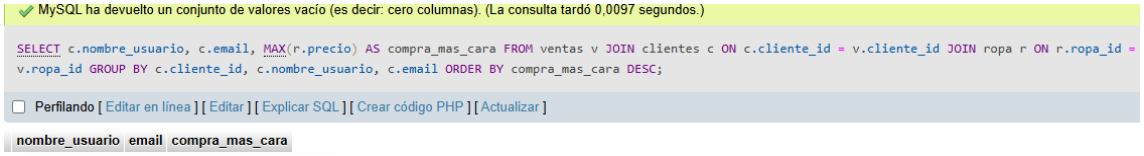
FROM ventas v

JOIN clientes c ON c.cliente_id = v.cliente_id

JOIN ropa r ON r.ropa_id = v.ropa_id

GROUP BY c.cliente_id, c.nombre_usuario, c.email

ORDER BY compra_mas_cara DESC;



Por si no funciona agregamos este código:

CREATE TABLE IF NOT EXISTS ventas_ropa (

venta_id INT AUTO_INCREMENT PRIMARY KEY,

cliente_id INT NOT NULL,

ropa_id INT NOT NULL,

fecha_venta DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,

FOREIGN KEY (cliente_id) REFERENCES clientes(cliente_id) ON DELETE CASCADE,

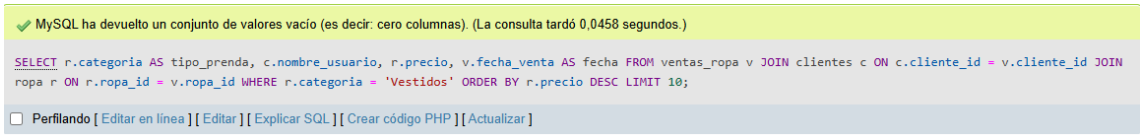
FOREIGN KEY (ropa_id) REFERENCES ropa(ropa_id) ON DELETE CASCADE

) ENGINE=InnoDB;

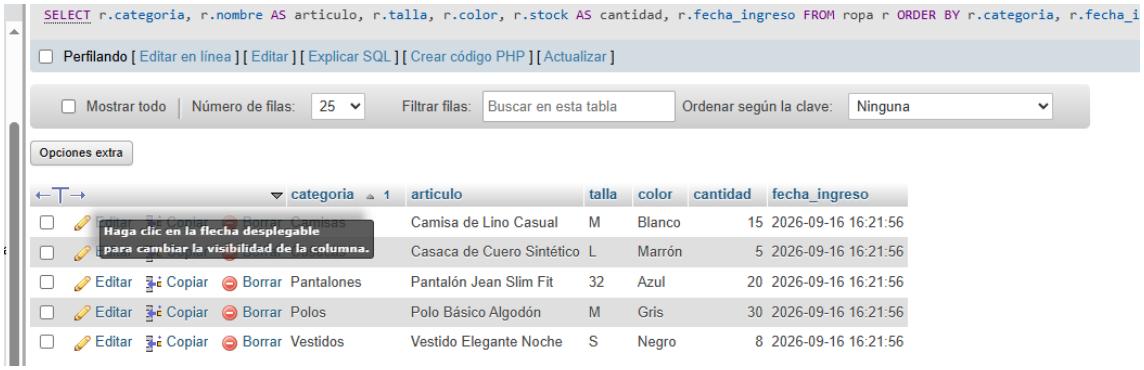
Luego

SELECT

```
    r.categoria AS tipo_prenda,
    c.nombre_usuario,
    r.precio,
    v.fecha_venta AS fecha
FROM ventas_ropa v
JOIN clientes c ON c.cliente_id = v.cliente_id
JOIN ropa r ON r.ropa_id = v.ropa_id
WHERE r.categoria = 'Vestidos'
ORDER BY r.precio DESC
LIMIT 10;
```

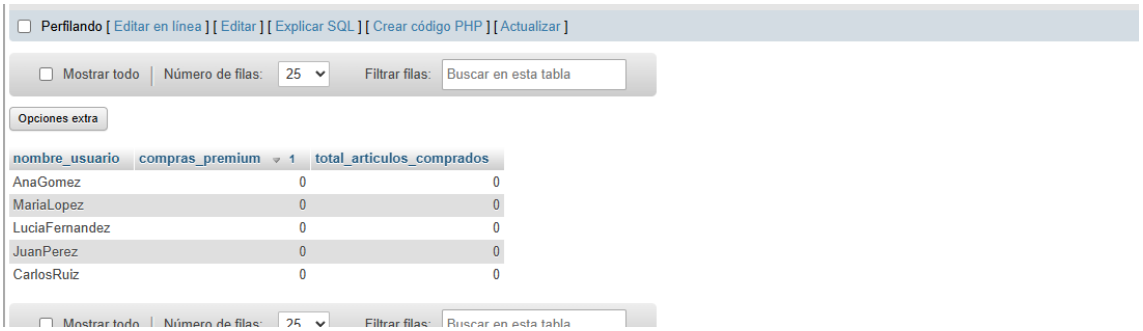



```
SELECT
    r.categoria,
    r.nombre AS articulo,
    r.talla,
    r.color,
    r.stock AS cantidad,
    r.fecha_ingreso
FROM ropa r
ORDER BY r.categoria, r.fecha_ingreso;
```




```
SELECT
```

```
    c.nombre_usuario,
    SUM(CASE WHEN r.precio > 50.00 THEN 1 ELSE 0 END) AS compras_premium,
    COUNT(v.venta_id) AS total_articulos_comprados
FROM clientes c
LEFT JOIN ventas v ON v.cliente_id = c.cliente_id
LEFT JOIN ropa r ON r.ropa_id = v.ropa_id
GROUP BY c.cliente_id, c.nombre_usuario
ORDER BY compras_premium DESC;
```



Paso 4 · Procedimiento: subir de nivel

Asegurarnos de las columnas en la tabla

```
ALTER TABLE clientes
ADD COLUMN IF NOT EXISTS nivel INT NOT NULL DEFAULT 1,
ADD COLUMN IF NOT EXISTS puntos INT NOT NULL DEFAULT 0;
```

Procedimiento de almacenado adaptado

```
USE yuri;


DROP PROCEDURE IF EXISTS subir_nivel_cliente;

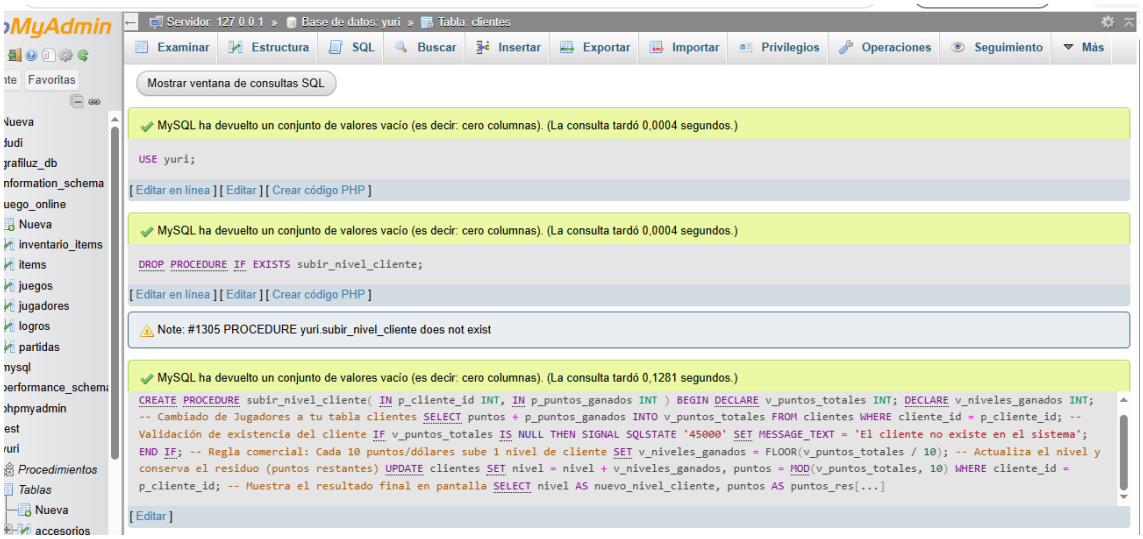

DELIMITER //


CREATE PROCEDURE subir_nivel_cliente(
    IN p_cliente_id INT,
    IN p_puntos_ganados INT
```

```
)
BEGIN
   DECLARE v_puntos_totales INT;
   DECLARE v_niveles_ganados INT;
}   SELECT puntos + p_puntos_ganados INTO v_puntos_totales
   FROM clientes WHERE cliente_id = p_cliente_id;
   IF v_puntos_totales IS NULL THEN
      SIGNAL SQLSTATE '45000'
         SET MESSAGE_TEXT = 'El cliente no existe en el sistema';
   END IF;

   SET v_niveles_ganados = FLOOR(v_puntos_totales / 10);
   UPDATE clientes
   SET nivel = nivel + v_niveles_ganados,
      puntos = MOD(v_puntos_totales, 10)
   WHERE cliente_id = p_cliente_id;
   SELECT nivel AS nuevo_nivel_cliente, puntos AS puntos_restantes
   FROM clientes WHERE cliente_id = p_cliente_id;
END //

DELIMITER ;
```


MyAdmin
Servidor: 127.0.0.1 » Base de datos: yuri » Tabla: clientes
Examinar
Estructura
SQL
Buscar
Insertar
Exportar
Importar
Privilegios
Operaciones
Seguimiento
Más
Favoritas
Nueva
dudi
grafiluz_db
nformation_schema
uego_online
Nueva
inventario_items
ítems
juegos
jugadores
logros
partidas
nysql
erformance_schema
hpmyadmin
est
uri
Procedimientos
Tablas
Nueva
accesorios
Mostrar ventana de consultas SQL
MySQL ha devuelto un conjunto de valores vacío (es decir: cero columnas). (La consulta tardó 0,0004 segundos.)
USE yuri;
[Editar en línea] [Editar] [Crear código PHP]
MySQL ha devuelto un conjunto de valores vacío (es decir: cero columnas). (La consulta tardó 0,0004 segundos.)
DROP PROCEDURE IF EXISTS subir_nivel_cliente;
[Editar en línea] [Editar] [Crear código PHP]
Note: #1305 PROCEDURE yuri.subir_nivel_cliente does not exist
MySQL ha devuelto un conjunto de valores vacío (es decir: cero columnas). (La consulta tardó 0,1281 segundos.)
CREATE PROCEDURE subir_nivel_cliente(IN p_cliente_id INT, IN p_puntos_ganados INT) BEGIN DECLARE v_puntos_totales INT; DECLARE v_niveles_ganados INT;
-- Cambiado de Jugadores a tu tabla clientes SELECT puntos + p_puntos_ganados INTO v_puntos_totales FROM clientes WHERE cliente_id = p_cliente_id; --
Validación de existencia del cliente IF v_puntos_totales IS NULL THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'El cliente no existe en el sistema';
END IF; -- Regla comercial: Cada 10 puntos/dólares sube 1 nivel de cliente SET v_niveles_ganados = FLOOR(v_puntos_totales / 10); -- Actualiza el nivel y
conserva el residuo (puntos restantes) UPDATE clientes SET nivel = nivel + v_niveles_ganados, puntos = MOD(v_puntos_totales, 10) WHERE cliente_id =
p_cliente_id; -- Muestra el resultado final en pantalla SELECT nivel AS nuevo_nivel_cliente, puntos AS puntos_res[...]
[Editar]